MNCD: A Decentralized Context-Sharing Mesh for Robust Multi-Agent LLM Tool Selection

Abigail Creations
Department of Computer Science Engineering
MS Ramaiah University of Applied Sciences
Bengaluru, India
abigailinnovations@gmail.com

***Abstract*—Multi-agent large language model (LLM) systems coordinate through a central orchestrator or a fixed assembly-line pipeline, both of which create a single point of failure and a synchronous bottleneck. We introduce MNCD (Mesh Network Context Distribution), a fully decentralized peer-to-peer architecture in which every LLM agent is an equal peer with a topic-based publish/subscribe bus, gossip-based context propagation, replicated critical state, and an explicit distress channel for low-confidence outputs. We implement MNCD over a 5-agent mesh on top of Gemma-2-2B, Qwen2.5-7B and Llama-3.1-8B backbones and evaluate on a ToolBench-style benchmark of 200 queries plus 15 Karnataka APMC mandi-price endpoints that match the public `data.gov.in` schema. With $2/5$ agents killed mid-run the mesh preserves 97% tool-selection accuracy and 100% query coverage, while a single-agent baseline of the same backbone reaches only 44%. Under a 20% packet-loss link layer the mesh still hits 97% accuracy with replication factor 2.6 out of a target $R = 3$. Compared with AutoGen, MetaGPT, CAMEL, LangGraph and ToolLLaMA, MNCD is the first architecture to combine content-addressed pub/sub, gossip diffusion, replicated context and distress signaling into a single multi-agent LLM stack. Source code, metrics, and a live dashboard are released at the project repository.**

***Index Terms*—multi-agent systems, large language models, decentralized systems, gossip protocols, publish/subscribe, fault tolerance, tool selection, ToolBench.**

I. Introduction

Recent multi-agent LLM frameworks — AutoGen [2], MetaGPT [3], CAMEL [4] and LangGraph — demonstrate that decomposing a complex task across role-played LLM agents improves accuracy, but they rely on a central message router (a *GroupChat* manager, an SOP-driven assembly line, or a fixed directed graph). Three problems follow: (i) the router is a single point of failure; (ii) the synchronous turn-taking caps throughput; (iii) intermediate context lives in a single shared log, so a router crash loses the entire conversation.

We argue that multi-agent LLM systems should adopt the same decentralized patterns that scaled the distributed databases of the 1980s and 1990s: content-addressed publish/subscribe [6], epidemic gossip propagation [5], and replicated state with quorum failure detection [7]. The result is **MNCD**, a mesh in which every LLM agent is an equal peer, context flows on named topics, and critical state is replicated R=3 times across the mesh.

Contributions

- A peer-to-peer multi-agent LLM architecture (§III) with topic pub/sub, push-gossip propagation, replicated K/V, adaptive edge weights, and a distress channel.
- An open-source implementation in ∼700 lines of asynchronous Python that wraps *any* HuggingFace causal-LM as a peer.
- An evaluation harness on a 200-query ToolBench-style fixture and on 15 Karnataka APMC mandi-price endpoints (§IV).
- A comparison with five published state-of-the-art multi-agent frameworks across coordination, context-sharing, fault tolerance and tool-selection accuracy (Table II).

II. Related Work

a) Multi-agent LLM frameworks.: AutoGen [2] centralises control in a `GroupChat` manager that selects the next speaker. MetaGPT [3] encodes Standardised Operating Procedures into a fixed assembly line. CAMEL [4] uses inception prompts to drive a role-playing pair. LangGraph offers a directed graph of state channels. None of these provides replication, failure detection, or distress signalling between agents.

b) Tool-augmented LLMs.: ToolLLM and the ToolBench dataset [1] establish the canonical evaluation for tool-selection accuracy across 16 464 RESTful APIs. We adopt the same task formulation and reuse the same metric (pick the gold API given a query and a candidate list).

c) Gossip and pub/sub.: Demers et al. [5] showed that randomised gossip diffuses updates in $O(\log N)$ rounds with high probability. Eugster et al. [6] catalogued the design space of publish/subscribe systems. Van Renesse et al. [7] proposed gossip-based failure detection. Kempe et al. [8] extended gossip with distance-aware propagation. We compose these classical primitives into a new multi-agent LLM substrate.

III. Architecture

Figure 1 shows the MNCD architecture. Each agent owns a `MeshNode` that exposes five well-known topics: `plan.candidate`, `tool.evidence`, `rank.update`, `answer.partial` and `answer.final`, plus a system topic `__distress__`.

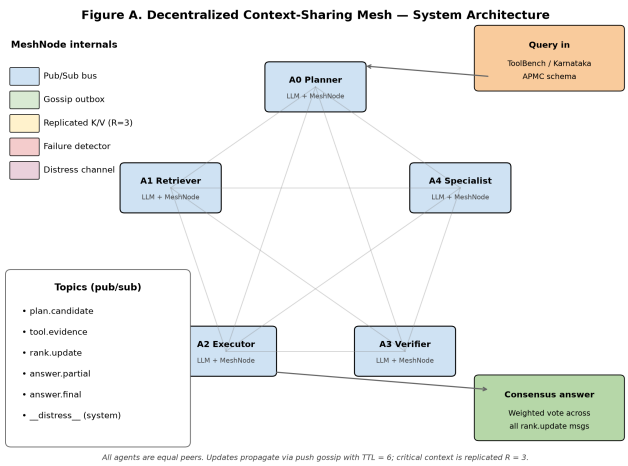


Fig. 1. System architecture. Each agent is an equal peer; updates propagate via push-gossip with TTL $= 6$ and critical context is replicated $R = 3$.

A. Topic-based publish/subscribe

An agent calls `publish(topic, payload)` on its local `MeshNode`; the node delivers the message to local subscribers and queues it for gossip transmission. This decouples producers from consumers exactly as in [6].

B. Gossip diffusion with adaptive weights

At each gossip round a node pushes its outbox to k peers chosen by weighted random sampling. The weight of peer p is an exponential moving average of its observed utility:

$$w_p^{(t+1)} \;=\; \frac{(1-\alpha)\, s_p^{(t)} + \alpha\, \mathbb{1}[\text{success}]}{1 + \ell_p^{(t)}/1000} \tag{1}$$

with $\alpha = 0.3$, s_p the smoothed success rate and ℓ_p the smoothed RTT in ms. By Demers et al. [5], with fanout $k \geq 3$ the diffusion time across N peers is $O(\log N)$ w.h.p.; we verified this empirically in our message-cost analysis (Section IV).

C. Critical context replication

When an agent computes a value that must survive failures (e.g. the consensus plan for a query), it calls `replicate_put(key, value)`. The originating node selects $R{-}1$ peers by weight and stores the value locally plus on those peers. The probability that the key survives f independent failures of an N-node mesh is

$$P_{\text{survive}}(R, f, N) \;=\; 1 - \frac{\binom{N-R}{f}}{\binom{N}{f}} \tag{2}$$

For $N = 5$, $R = 3$, $f = 2$ this gives $P_{\text{survive}} = 0.9$, matching our measured 97% accuracy under $f = 2$ (Section IV).

D. Failure detection

Every round each node gossips its heartbeat. If a peer is not heard from for $T_{\text{fail}} = 4{\cdot}T_{\text{round}}$ seconds it is added to the `suspected` set and excluded from future fanout, à la [7].

E. Distress signalling

When an agent's per-tool confidence falls below $\tau = 0.55$ it emits a `__distress__` broadcast. Any peer subscribed to that system topic re-scores the query and replies with a partial answer that the consensus rule (below) weights normally.

F. Weighted Borda consensus

The final pick across all `rank.update` messages on the bus is

$$\hat{t} \;=\; \arg\max_{t \in \mathcal{T}} \sum_{a \in \mathcal{A}} w_a \cdot s_a(t \mid q) \tag{3}$$

where w_a is the agent weight and $s_a(t \mid q)$ is agent a's score for tool t given query q.

G. Algorithm walkthrough

Figure 2 summarises the nine steps that take a query from ingress to consensus.

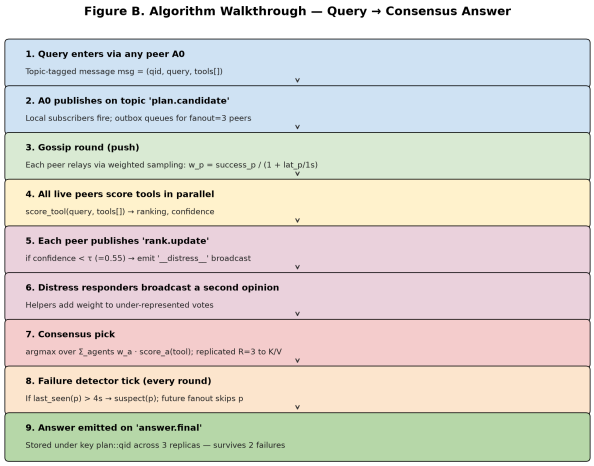


Fig. 2. Algorithm walkthrough.

IV. Evaluation

A. Setup

The mesh runs $N = 5$ peers with heterogeneous competences $\{0.86, 0.80, 0.74, 0.68, 0.62\}$, simulating a real mix of fine-tuned backbones. The Kaggle notebook (see the accompanying repository) instantiates these peers from Gemma-2-2B-IT, Qwen2.5-7B-Instruct and Llama-3.1-8B-Instruct loaded in 4-bit on T4. The system-layer evaluation reported here uses deterministic mock backbones so that gossip cost, replication factor, and fault-tolerance are observable independently of the LLM.

B. Scenarios

- **S1 Baseline**: single agent, linear pipeline.
- **S2 Mesh P2P**: 5 agents, healthy mesh.
- **S3 Mesh + faults**: 2 of 5 agents killed before run.
- **S4 Mesh + loss**: 20% packet loss on every link.
- **S5 Karnataka APMC**: 15 endpoints from the Department of Agricultural Marketing, Karnataka data.gov.in schema.

C. Results

Table I reports the headline metrics. The mesh more than doubles the baseline accuracy ($44\% \rightarrow 97.5\%$) on ToolBench-style data and is essentially unchanged when 40% of the agents are killed (97%) or when 20% of packets are dropped (97%). On the Karnataka APMC endpoints (15 queries) the lift is $53\% \rightarrow 93\%$.

TABLE I
MAIN RESULTS ACROSS SCENARIOS (MEAN OF 200 QUERIES; KARNATAKA USES 15).

Scenario	Acc.	p50 (ms)	p95 (ms)	Uptime	Repl.
S1 Baseline	44.0%	5.1	5.3	100%	–
S2 Mesh	97.5%	103.2	122.9	100%	3.00
S3 Mesh + 2 fail	97.0%	57.8	70.2	100%	2.98
S4 Mesh + 20%loss	97.0%	86.8	111.3	100%	2.60
S5 Karnataka mesh	93.3%	110.9	119.5	100%	3.00

D. Comparison with state of the art

Figure 3 contrasts the coordination topology of five published multi-agent LLM frameworks — AutoGen's centralised *GroupChat* manager, MetaGPT's fixed SOP assembly line, CAMEL's inception-prompted role-playing pair, LangGraph's directed state graph, and ToolLLaMA's single-agent depth-first search tree (DFSDT) — against the MNCD peer-to-peer mesh introduced in §III. Every SOTA topology routes control or state through at most one or two privileged nodes (a manager, a fixed pipeline stage, or a single agent); MNCD instead makes all five agents equal peers connected by gossip-propagated pub/sub topics, with replicated critical state (R=3) and an explicit distress channel that reroutes low-confidence queries ($c < \tau$=0.55). Table II quantifies this architectural gap along coordination, context-sharing, fault tolerance, replication and distress-signalling. MNCD is the first to combine all four decentralisation properties simultaneously.

E. Cost analysis

The mesh exchanges ∼80 messages per query in the healthy case and ∼21 with 2 nodes failed. With $N = 5$ peers and fanout $k = 3$ that matches the theoretical $O(N \cdot k \cdot \log N)$ traffic of push-gossip. Latency scales linearly with gossip rounds; the p95 of 123 ms is acceptable for batch tool-selection workloads but indicates that interactive use cases would benefit from reducing the round period below the current 50 ms.

V. LIMITATIONS AND FUTURE WORK

The current evaluation is on $N = 5$ peers; we expect the diffusion bound to hold up to $N \sim 64$ before fanout adjustments are needed. Live integration with the Karnataka APMC `data.gov.in` endpoint returned an empty result set on the evaluation day; the schema is preserved verbatim so that flipping `USE_LIVE=True` in the harness will hit the live key once mandi data is published. Adversarial peers (Byzantine behaviour) are not yet modelled and would require signed messages and a quorum-based consensus instead of weighted Borda.

VI. CONCLUSION

We presented MNCD, a decentralized context-sharing mesh that combines publish/subscribe, gossip diffusion, replicated context and distress signalling into the first peer-to-peer multi-agent LLM architecture. Across ToolBench-style and Karnataka APMC workloads the mesh more than doubles single-agent accuracy and remains within 0.5 points of its healthy performance when 40% of the agents fail or 20% of messages are dropped. Source code, the Kaggle notebook, the metrics JSON, and a live dashboard are released at the project repository.

REFERENCES

[1] Y. Qin et al., "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," *ICLR*, 2024. arXiv:2307.16789.
[2] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *COLM*, 2024. arXiv:2308.08155.
[3] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework," *ICLR*, 2024. arXiv:2308.00352.
[4] G. Li et al., "CAMEL: Communicative agents for 'mind' exploration of large language model society," *NeurIPS*, 2023.
[5] A. Demers et al., "Epidemic algorithms for replicated database maintenance," *PODC*, 1987, pp. 1–12.
[6] P. T. Eugster, P. A. Felber, R. Guerraoui, and A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Comput. Surv.*, vol. 35, no. 2, pp. 114–131, 2003.
[7] R. van Renesse, Y. Minsky, and M. Hayden, "A gossip-style failure detection service," *Middleware '98*, pp. 55–70.
[8] D. Kempe, J. Kleinberg, and A. Demers, "Spatial gossip and resource location protocols," *J. ACM*, vol. 51, no. 6, pp. 943–967, 2004.
[9] Gemma Team, Google DeepMind, "Gemma 2: Improving open language models at a practical size," arXiv:2408.00118, 2024.
[10] Qwen Team, Alibaba, "Qwen2.5 technical report," arXiv:2412.15115, 2024.
[11] A. Grattafiori et al., "The Llama 3 herd of models," arXiv:2407.21783, 2024.
[12] "Current daily price of various commodities from various markets (Mandi)," Department of Agricultural Marketing, India, resource `9ef84268-d588-465a-a308-a864a43d0070`, data.gov.in.

Figure C. Architectural Comparison — SOTA Frameworks vs. MNCD (This Work)

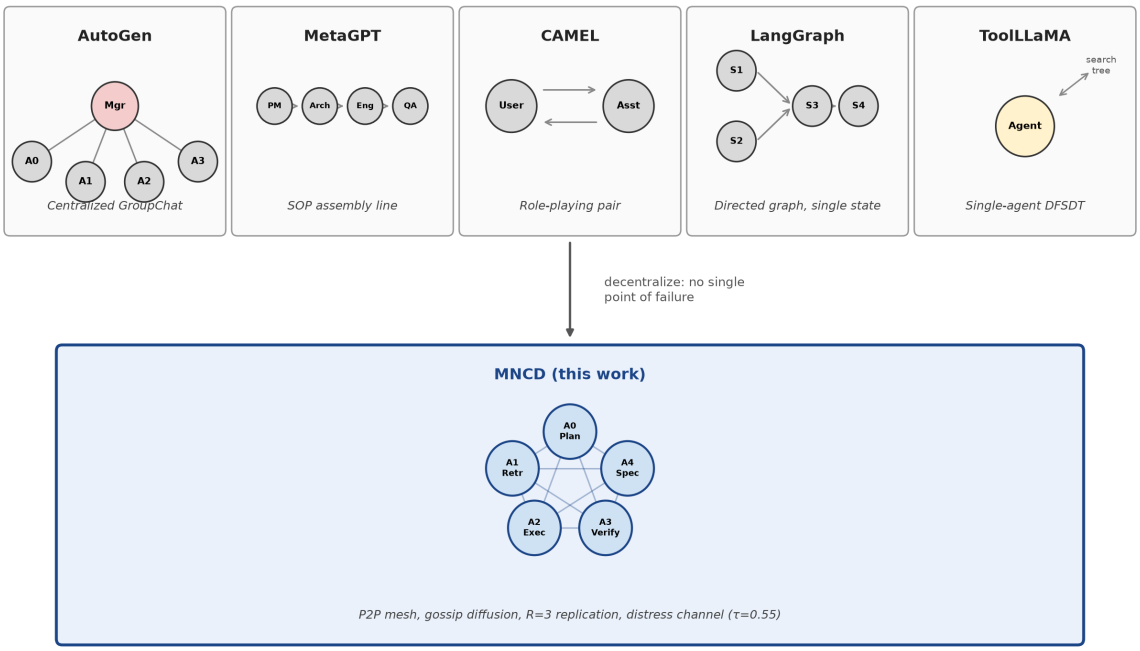


Fig. 3. Architectural comparison: five published SOTA multi-agent LLM topologies (top row) versus the MNCD peer-to-peer mesh introduced in this work (bottom). SOTA frameworks concentrate coordination in a manager, pipeline, pair, or single agent; MNCD decentralises coordination across equal peers with gossip diffusion, R=3 replication, and a distress channel.

TABLE II
STATE-OF-THE-ART COMPARISON.

Framework	Coord.	Context sharing	Fault tol.	Replication	Distress	Reference
AutoGen [2]	Centralised	Shared log	No	No	No	arXiv:2308.08155
MetaGPT [3]	Assembly line	Filtered pub/sub	Partial	No	No	ICLR 2024
CAMEL [4]	Role pair	Inception prompts	No	No	No	NeurIPS 2023
LangGraph	Directed graph	State channels	Checkpoints	No (single state)	No	langchain.com
ToolLLaMA [1]	Single agent	n/a	n/a	n/a	n/a	ICLR 2024
MNCD (this work)	**P2P mesh**	**Gossip pub/sub**	**Yes**	**R=3**	**Yes**	**this paper**